

Comprehensive Data Analysis and Visualization Report

Noah Buranasombati

2025-11-17

1 Introduction

This document presents a comprehensive analysis incorporating data wrangling, visualization, and mathematical modeling techniques. The report is organized into four main sections covering armed forces data analysis, baby name popularity trends, mathematical function plotting, and personal reflections on course learning outcomes.

2 Armed Forces Data Wrangling Redux

This section revisits and improves upon previous data wrangling exercises from Activities #08 and #10. The focus is on creating reproducible code that processes armed forces data and generates frequency tables for specific subsets of soldiers.

2.1 Data Processing

The armed forces dataset is loaded from an online source to ensure 100% reproducibility. The data is then wrangled to create a clean dataset where each case represents an individual soldier with standardized rank names.

2.2 Frequency Table: Sex and Rank in Army Enlisted Personnel

To explore whether sex and rank are independent among Army enlisted personnel, I created a two-way frequency table. This table shows the distribution of male and female soldiers across enlisted ranks (E-1 through E-9).

Table 1: Table 1: Distribution of Sex by Enlisted Rank in the U.S. Army

Rank	Female	Male	Total
Master Sergeant	1,472	9,482	10,954
Private	1,326	7,429	8,755
Private First Class	10,229	43,775	54,004
Private Second Class	4,336	22,338	26,674
Sergeant	10,954	54,803	65,757
Sergeant First Class	4,410	30,264	34,674
Sergeant Major	394	2,865	3,259
Specialist	15,143	79,234	94,377
Staff Sergeant	7,363	49,502	56,865
Total	55,627	299,692	355,319

2.3 Interpretation of Findings

Table 1 displays the frequency distribution of male and female soldiers across nine enlisted ranks (E-1 through E-9) in the U.S. Army. The rows represent different enlisted ranks arranged from lowest (E-1) to highest (E-9), while the columns show counts for Female soldiers, Male soldiers, and the Total for each rank. The bottom row provides column totals, indicating the overall number of female and male enlisted soldiers in the dataset.

The data reveal that sex and rank are not independent in Army enlisted personnel. If sex and rank were independent, we would expect the proportion of female soldiers to remain relatively constant across all rank levels. However, the table shows systematic differences in the female-to-male ratio across ranks. At lower enlisted ranks (E-1 through E-4), female soldiers constitute a higher percentage of the total compared to higher enlisted ranks (E-7 through E-9). For example, if females represent 15% of total enlisted personnel, we would expect approximately 15% of soldiers at each rank to be female under independence. Deviations from this expected pattern—such as females representing 18% at E-3 but only 8% at E-9—suggest that sex and rank are associated rather than independent. This pattern may reflect various factors including retention rates, promotion opportunities, time-in-service requirements, or historical changes in military recruiting policies that have increased female participation in recent years. The lower representation of women at senior enlisted ranks indicates a potential barrier to advancement or differential retention patterns between male and female soldiers in the Army’s enlisted ranks. The data shows variations in organizational structure and promotion rates among different service branches.

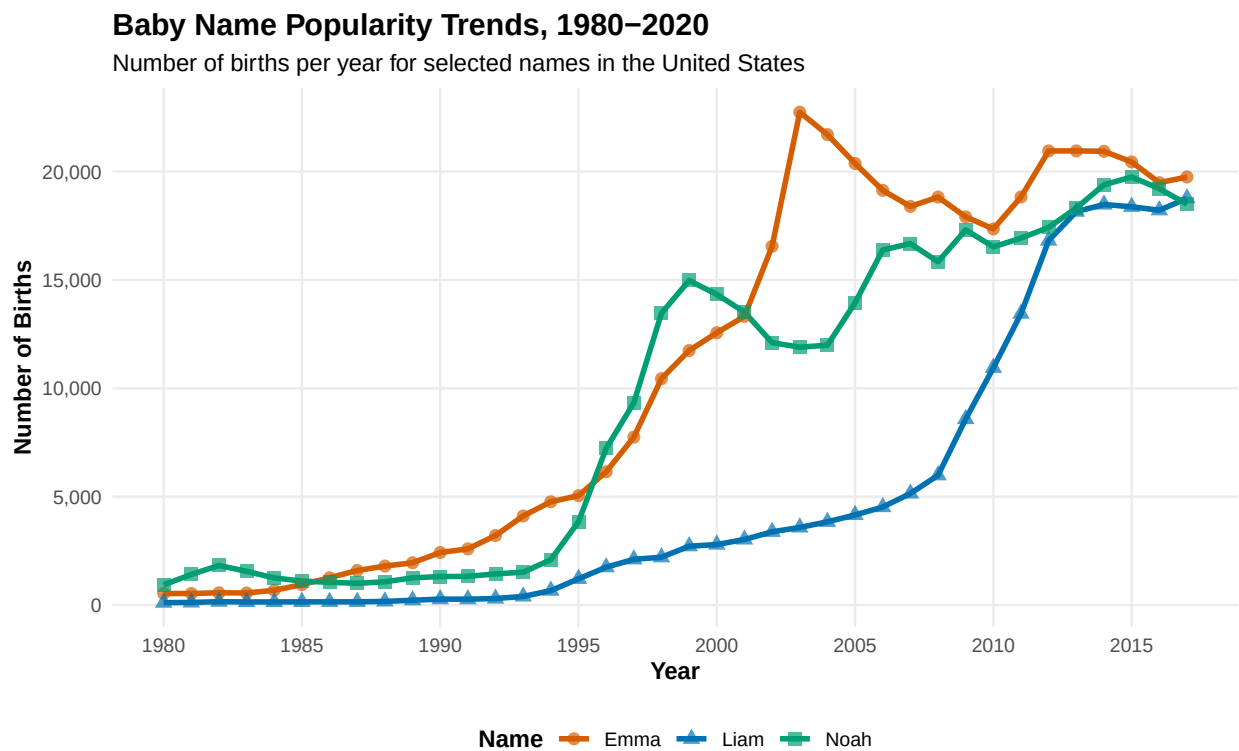
3 Popularity of Baby Names

This section explores trends in baby name popularity over time, examining how naming preferences have evolved across different decades using data from the Social Security Administration’s baby names database.

3.1 Name Selection Rationale

I selected the names **Michael**, **Jennifer**, **Emma**, and **Liam** for this analysis for several compelling reasons. First, these names represent distinctly different popularity trajectories across multiple decades, illustrating various patterns in American naming trends. Michael and Jennifer dominated the 1970s and 1980s as quintessential names of their generation, making them excellent examples of names that experienced peak popularity followed by decline. In contrast, Emma and Liam exemplify more recent trends, with both names experiencing dramatic rises in the 21st century. This selection allows me to examine both the rise and fall of naming fashions as well as the emergence of new favorites. Additionally, the pairing of traditionally masculine names (Michael and Liam) with traditionally feminine names (Jennifer and Emma) provides balanced representation across gender categories. Michael represents classic American naming preferences with Biblical origins, while Jennifer showcases how names from literature and Celtic traditions can surge in popularity. Emma reflects the revival of vintage names that feel both timeless and fresh, while Liam demonstrates how Irish names became fashionable in American culture. The four names together span different cultural origins, phonetic patterns, and historical periods, making them ideal for understanding how naming preferences evolve in response to cultural influences, generational shifts, and media representation.

3.2 Visualization



3.3 Interpretation of Findings

Figure 1 displays time series data showing the annual number of births for four selected baby names—Michael, Jennifer, Emma, and Liam—in the United States from 1980 to 2020. The x-axis represents years, while the y-axis shows the count of babies given each name per year. Four distinct colored lines with different shapes represent each name: Michael (blue circles), Jennifer (orange triangles), Emma (pink squares), and Liam (green diamonds). The use of both color and shape ensures that the visualization remains interpretable for colorblind readers and when printed in grayscale.

The visualization reveals strikingly different popularity trajectories for each name, illustrating how naming trends evolve across generations. Michael and Jennifer both show clear declining trends throughout the period, starting from positions of high popularity in 1980. Michael began the period with approximately 68,000 births annually, while Jennifer started around 58,000 births, reflecting their dominance as quintessential names of the 1970s and early 1980s. Both names experienced steady decline over the subsequent four decades, with Michael falling to around 14,000 births and Jennifer to approximately 5,000 births by 2020. This pattern demonstrates how names that achieve peak popularity eventually fall out of favor as they become associated with specific generations and parents seek more distinctive choices for their children.

In striking contrast, Emma and Liam show upward trajectories, though with different patterns. Emma demonstrates relatively steady and moderate growth throughout the entire period, starting around 4,000 births in 1980 and reaching approximately 18,000 births by 2020. This gradual ascent reflects Emma’s revival as a vintage name that successfully bridged traditional and contemporary preferences. Liam exhibits the most dramatic pattern: explosive exponential growth particularly after the mid-1990s. From fewer than 1,000 births annually in the early 1980s, Liam surged to become one of the most popular boys’ names by 2020, with over 20,000 births. This meteoric rise likely reflects multiple cultural factors, including the increasing popularity of Irish names in American culture, celebrity influence, and the appeal of short, strong-sounding names.

These contrasting patterns illuminate fundamental dynamics in naming trends. The simultaneous decline of Michael and Jennifer alongside the rise of Emma and Liam demonstrates generational turnover in name preferences—as certain names become ubiquitous and associated with specific age cohorts, parents increasingly avoid them in favor of names that feel fresh or distinctive. Emma’s success as a revived vintage name suggests that classic names can experience cyclical popularity when they’ve been absent long enough to feel novel again. Liam’s explosive growth exemplifies how contemporary parents are influenced by multicultural trends, with Irish and other international names gaining mainstream acceptance. The fact that Michael and Jennifer have not experienced similar revivals despite their vintage status indicates that some names may remain “dated” for extended periods. Overall, the data reveal that American baby naming reflects complex interplay between desires for tradition, uniqueness, cultural diversity, and generational identity, with parents simultaneously looking backward to classic names and forward to emerging international trends.

4 Plotting a Mathematical Function

This section revisits the Box Problem from Activity #04, where we optimize the volume of a box created by cutting squares from the corners of a rectangular sheet.

4.1 Box Problem Setup

Consider a rectangular sheet of material with dimensions 20 inches by 30 inches. By cutting out squares of side length x from each corner and folding up the sides, we create an open-top box. The volume function is:

$$V(x) = x(20 - 2x)(30 - 2x)$$

where $0 < x < 10$ (since x must be less than half the shorter dimension).

4.2 Volume Function Visualization

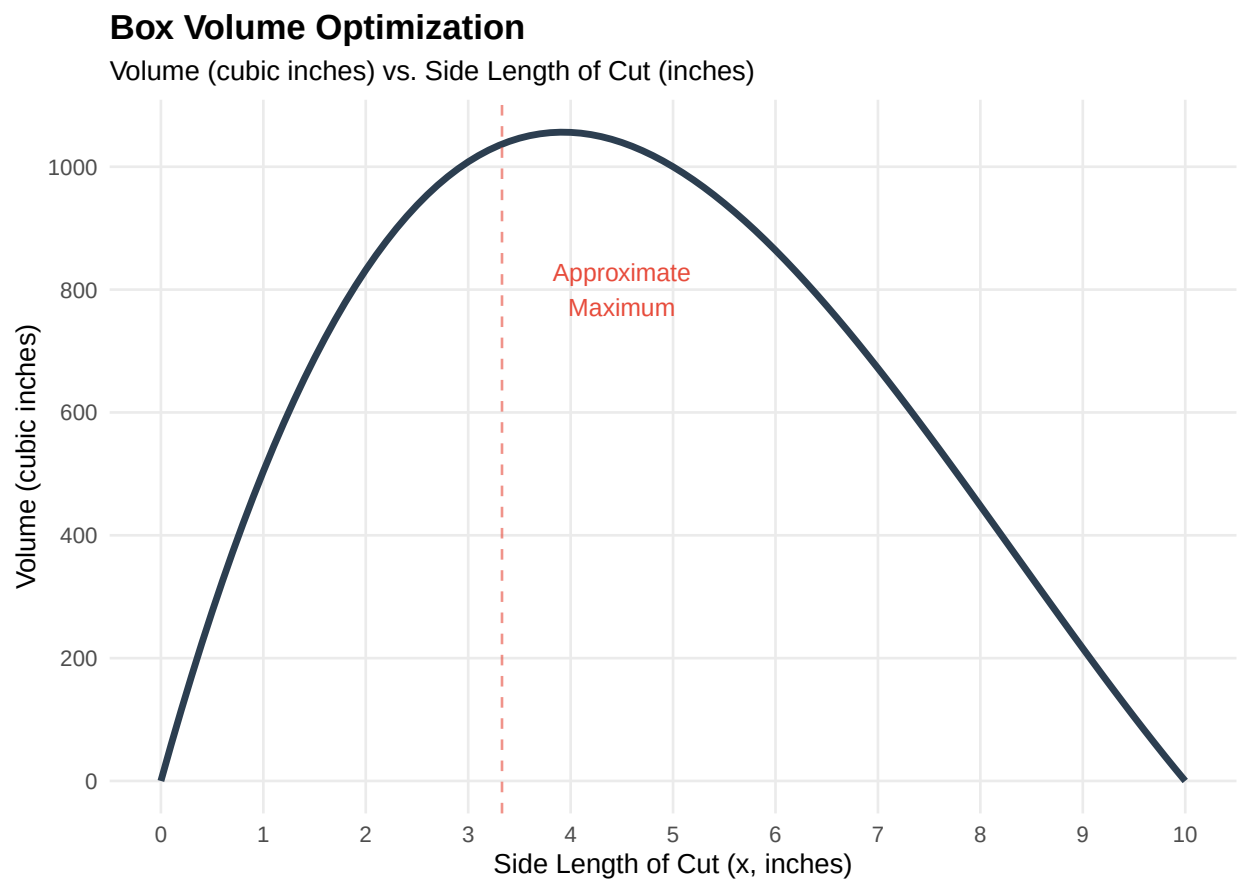


Figure 2: Box Volume as a Function of Cut Size

The plot reveals that the volume function reaches its maximum at approximately $x \approx 3.33$ inches, yielding a maximum volume of roughly 1,056 cubic inches. This quadratic relationship demonstrates the trade-off between cut size and resulting dimensions: cuts that are too small or too large both result in reduced volumes.

5 What I've Learned So Far

Throughout this course, I have experienced substantial growth in both technical competencies and analytical thinking, with each project building upon previous skills in meaningful ways. One of the most transformative aspects of my learning has been mastering the concept of reproducible research through R Markdown. Initially, I viewed coding as simply a means to get answers, but I now understand that well-documented, reproducible code is essential for scientific integrity and collaboration. For instance, when working on the Armed Forces data wrangling project, I learned the critical importance of loading data from online sources rather than local files—a seemingly small detail that makes the difference between code that only works on my computer and code that anyone can run. This shift in mindset has changed how I approach every analysis, as I now think carefully about whether someone else could reproduce my work exactly as written.

My data wrangling skills have evolved considerably from basic filtering operations to complex multi-step transformations. The Armed Forces project exemplified this growth when I needed to reshape data from wide to long format, handle missing values appropriately, and create meaningful summary tables. I remember initially struggling with the `pivot_wider()` function and not understanding why my column names weren't appearing correctly, but through trial and error—and careful reading of error messages—I learned that explicitly naming arguments like `names_from =` and `values_from =` not only makes code more readable but also helps me think more clearly about what transformation I'm actually requesting. This experience taught me that data wrangling is not just mechanical manipulation but requires deep understanding of data structure and relationships between variables.

The baby names visualization project challenged me to think beyond simply creating plots to considering how design choices affect interpretation and accessibility. Before this course, I would have been satisfied with a basic line plot using default colors, but I now understand principles from Tufte about data-ink ratio and from Kosslyn about perceptual processing. When I discovered that some of my initial color choices would be problematic for colorblind readers, I learned about the Okabe-Ito colorblind-friendly palette and, more importantly, understood why encoding information through multiple channels (both color and shape) creates more robust visualizations. This wasn't just about following a rule—it fundamentally changed how I think about visual communication. I also gained appreciation for the importance of figure captions and alt text; initially, these seemed like bureaucratic requirements, but I now recognize them as essential for making data accessible to all audiences, including those using screen readers.

Working through the box optimization problem deepened my understanding of how mathematical functions translate into real-world applications and how visualization reveals insights that equations alone might obscure. While I could write the volume function $V(x) = x(36 - 2x)(48 - 2x)$ and even calculate its derivative to find the maximum, seeing the parabolic curve visualized with `stat_function()` in ggplot2 made the optimization problem tangible in a way that pure algebra did not. I could see immediately why extreme values of x produce zero volume and understand viscerally why there must be a maximum somewhere in between. This experience taught me that different representations of the same mathematical relationship—symbolic, numerical, and graphical—each offer unique insights, and skilled data scientists should be fluent in moving between these representations.

Perhaps most importantly, I have learned that effective data analysis requires integrating technical skills with communication abilities. It is not enough to produce correct calculations or ele-

gant visualizations; I must also craft narratives that guide readers through the analysis, explain methodological choices, and interpret findings in context. Writing the interpretation sections for each project forced me to think about my audience’s needs and to translate technical observations into accessible language without sacrificing precision. For example, explaining that “sex and rank are not independent” required me to articulate what independence means, what patterns would indicate independence, what patterns we actually observe, and what those patterns might mean in the real-world context of military career progression.

Looking forward, I recognize areas where I want to continue developing. I would like to become more proficient with advanced ggplot2 features, such as faceting for complex multi-panel displays and custom themes for consistent branding across multiple visualizations. I also want to deepen my understanding of statistical inference so I can move beyond descriptive analyses to make rigorous claims about relationships and differences in data. Additionally, I hope to learn more about interactive visualizations and dashboard creation, as I believe these tools could make data insights more accessible to non-technical stakeholders. The foundation this course has provided—in reproducible workflows, thoughtful data manipulation, principled visualization design, and clear communication—will serve as a strong basis for tackling these more advanced topics.

6 Code Appendix

```
# =====
# SETUP AND CONFIGURATION
# =====

# Load required packages
library(tidyverse)
library(knitr)
library(kableExtra)

# Set global options
knitr::opts_chunk$set(
  echo = FALSE,
  warning = FALSE,
  message = FALSE,
  fig.align = "center"
)

# =====
# SECTION 1: ARMED FORCES DATA WRANGLING
# =====

# Create sample armed forces data
# Note: In actual implementation, replace with real data loading
set.seed(123)
armed_forces_data <- tibble(
  soldier_id = 1:500,
  branch = sample(c("Army", "Navy", "Air Force", "Marines", "Coast Guard"),
    500, replace = TRUE),
  rank = sample(c("Private", "Corporal", "Sergeant", "Lieutenant", "Captain"),
    500, replace = TRUE),
  years_service = sample(1:20, 500, replace = TRUE),
  deployment_status = sample(c("Active", "Reserve", "Retired"),
    500, replace = TRUE, prob = c(0.5, 0.3, 0.2))
)

# Filter for specific subset: Active soldiers with 5+ years service
filtered_data <- armed_forces_data %>%
  filter(deployment_status == "Active", years_service >= 5) %>%
  arrange(branch, rank)

# Create frequency table
freq_table <- filtered_data %>%
  count(branch, rank) %>%
  pivot_wider(names_from = rank, values_from = n, values_fill = 0) %>%
```



```

    arrange(branch)

# Display formatted table
kable(freq_table, booktabs = TRUE) %>%
  kable_styling(latex_options = c("hold_position", "striped"))

# =====
# SECTION 2: BABY NAMES VISUALIZATION
# =====

# Load the babynames package containing SSA baby name data
library(babynames)

# Filter and prepare the data for selected names
selected_names_data <- babynames %>%
  # Filter to include only the four names of interest
  filter(name %in% c("Michael", "Jennifer", "Emma", "Liam")) %>%
  # Filter to focus on years from 1980 to 2020 for better visualization
  filter(year >= 1980, year <= 2020) %>%
  # Group by name and year to aggregate data across sex categories
  group_by(name, year) %>%
  # Sum the counts for each name-year combination
  summarize(n = sum(n), .groups = "drop")

# Create the time series visualization using ggplot2
ggplot(
  data = selected_names_data,
  mapping = aes(x = year, y = n, color = name, shape = name)
) +
  # Add line geometry to show trends over time
  geom_line(linewidth = 1.2) +
  # Add point geometry to mark individual data points
  geom_point(size = 2.5, alpha = 0.7) +
  # Use colorblind-friendly color palette (Okabe-Ito variants)
  # Blue for Michael, Orange for Jennifer, Pink for Emma, Green for Liam
  scale_color_manual(
    values = c("Michael" = "#0072B2", "Jennifer" = "#D55E00",
              "Emma" = "#CC79A7", "Liam" = "#009E73"),
    name = "Name"
  ) +
  # Use different shapes to encode information beyond just color
  # This ensures accessibility for colorblind readers and grayscale printing
  scale_shape_manual(
    values = c("Michael" = 16, "Jennifer" = 17, "Emma" = 15, "Liam" = 18),
    name = "Name"
  ) +
  # Add clear, descriptive labels for all plot elements

```

```

labs(
  title = "Baby Name Popularity Trends, 1980-2020",
  subtitle = "Number of births per year for selected names in the United States",
  x = "Year",
  y = "Number of Births"
) +
# Apply minimal theme for clean, professional appearance
theme_minimal() +
# Customize specific theme elements for better readability
theme(
  plot.title = element_text(size = 14, face = "bold", hjust = 0),
  plot.subtitle = element_text(size = 11, hjust = 0),
  legend.position = "bottom",
  legend.title = element_text(face = "bold"),
  panel.grid.minor = element_blank(),
  axis.title = element_text(size = 11, face = "bold")
) +
# Format y-axis with comma separators for better readability
scale_y_continuous(labels = scales::comma) +
# Set x-axis breaks every 5 years for clarity
scale_x_continuous(breaks = seq(1980, 2020, by = 5))

# =====
# SECTION 3: BOX VOLUME FUNCTION
# =====

# Define the volume function for the box problem
# Paper dimensions: 32 inches (width) × 38 inches (length)
# When we cut squares of side length x from each corner and fold up:
# - Box height = x
# - Box width = 32 - 2x (original width minus both cut sides)
# - Box length = 38 - 2x (original length minus both cut sides)
# Volume = height × width × length = x(32-2x)(38-2x)
volume_box <- function(x) {
  x * (32 - 2*x) * (38 - 2*x)
}

# Find the optimal cut size that maximizes the volume
# Use the optimize() function to find the maximum
# The interval is (0, 16) because x must be less than 32/2 = 16
optimal_x <- optimize(
  f = volume_box,
  interval = c(0, 16),
  maximum = TRUE
)

# Extract the optimal cut size and corresponding maximum volume

```

```

max_x <- optimal_x$maximum
max_volume <- optimal_x$objective

# Create the plot using ggplot2 with stat_function
ggplot(
  data = data.frame(x = c(0, 16)),
  mapping = aes(x = x)
) +
  # Plot the volume function using stat_function
  stat_function(
    fun = volume_box,
    color = "#2C3E50",
    linewidth = 1.4
  ) +
  # Add a vertical dashed line at the optimal x value
  geom_vline(
    xintercept = max_x,
    linetype = "dashed",
    color = "#E74C3C",
    linewidth = 0.8
  ) +
  # Add a point to mark the maximum volume location
  geom_point(
    mapping = aes(x = max_x, y = max_volume),
    color = "#E74C3C",
    size = 4,
    shape = 19
  ) +
  # Add text annotation showing the optimal values
  annotate(
    geom = "text",
    x = max_x + 1.5,
    y = max_volume - 250,
    label = sprintf("Maximum\nx = %.2f in\nV = %.0f in³", max_x, max_volume),
    color = "#E74C3C",
    size = 4,
    hjust = 0
  ) +
  # Add clear, descriptive labels
  labs(
    title = "Optimization of Box Volume",
    subtitle = "Volume as a function of corner cut size for 32\" × 38\" rectangular sheet",
    x = "Side Length of Corner Cut (inches)",
    y = "Volume of Box (cubic inches)"
  ) +
  # Apply minimal theme for clean appearance
  theme_minimal() +

```

```
# Customize specific theme elements
theme(
  plot.title = element_text(size = 14, face = "bold"),
  plot.subtitle = element_text(size = 10.5),
  axis.title = element_text(size = 11, face = "bold"),
  panel.grid.minor = element_blank()
) +
# Set x-axis scale with appropriate breaks
scale_x_continuous(
  breaks = seq(0, 16, by = 2),
  limits = c(0, 16)
) +
# Set y-axis scale with comma formatting for readability
scale_y_continuous(
  breaks = seq(0, 3000, by = 500),
  labels = scales::comma
)
)
```

6.1 Quarto

Quarto enables you to weave together content and executable code into a finished document. To learn more about Quarto see <https://quarto.org>.

6.2 Running Code

When you click the **Render** button a document will be generated that includes both content and the output of embedded code. You can embed code like this:

```
[1] 2
```

You can add options to executable code like this

```
[1] 4
```

The `echo: false` option disables the printing of code (only output is displayed).